

Module 5

Memory Management

Virtual Memory

- Virtual memory is a technique that allows the execution of processes that
- are not completely in memory.
- One major advantage of this scheme is that programs can be larger than physical memory.
- virtual memory abstracts main memory into an extremely large, uniform array of storage, separating logical memory as viewed by the programmer from physical memory.
- It frees programmers from the concerns of memory-storage limitations.

Virtual Memory

- **Virtual memory** involves the separation of logical memory as perceived by developers from physical memory.
- This separation allows an extremely large virtual memory to be provided for programmers when only a smaller physical memory is available.
- programming the problem that is to be solved.
- The **virtual address space** of a process refers to the logical (or virtual) view of how a process is stored in memory. The process begins at a certain logical address—say, address 0—and exists in contiguous memory.

Virtual Memory

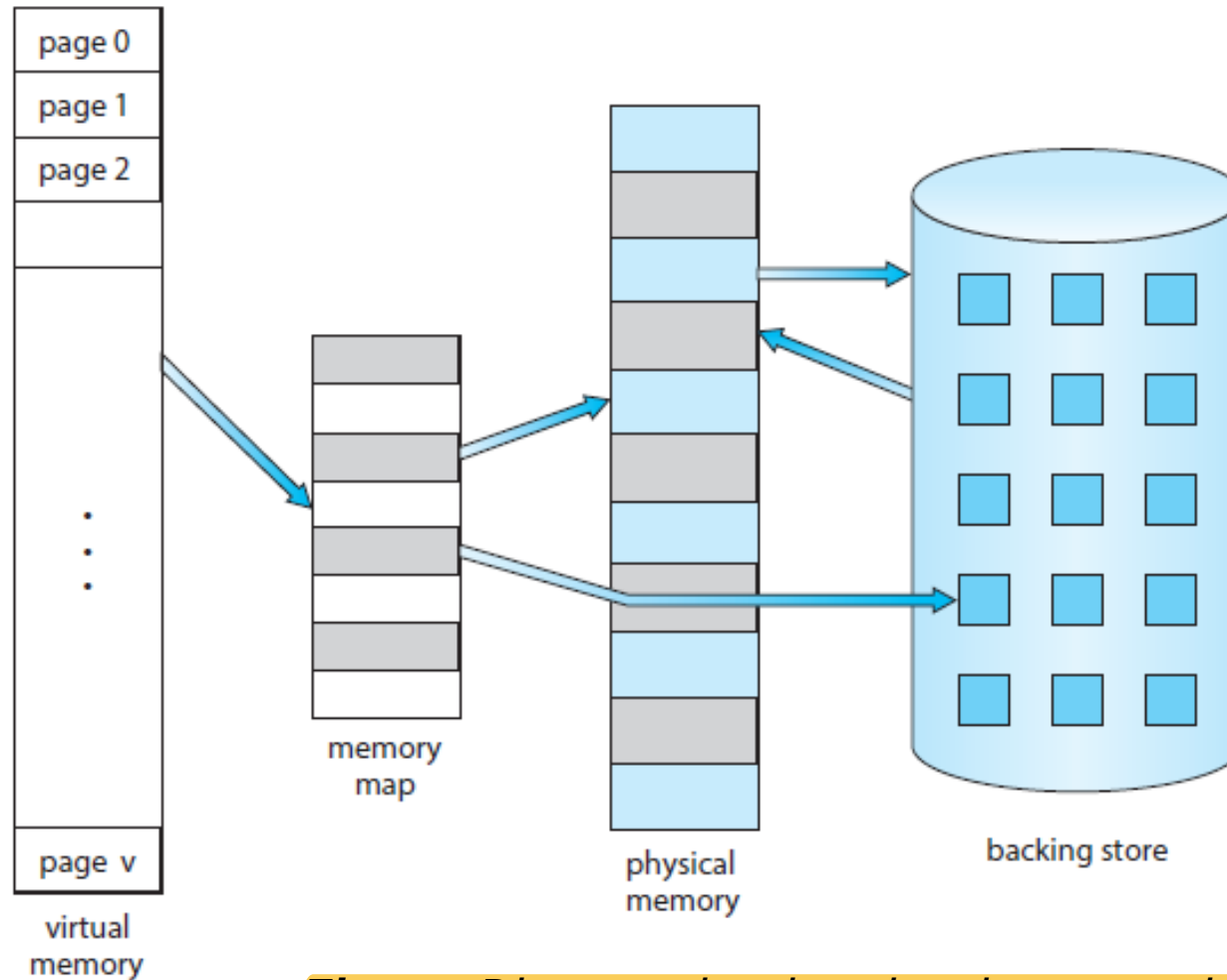


Figure: Diagram showing virtual memory that is larger than physical memory

Virtual Address Space

- Virtual address spaces that include holes are known as **sparse address spaces**.
- Using a sparse address space is beneficial because the holes can be filled as the stack or heap segments grow or if we wish to dynamically link libraries during program execution.

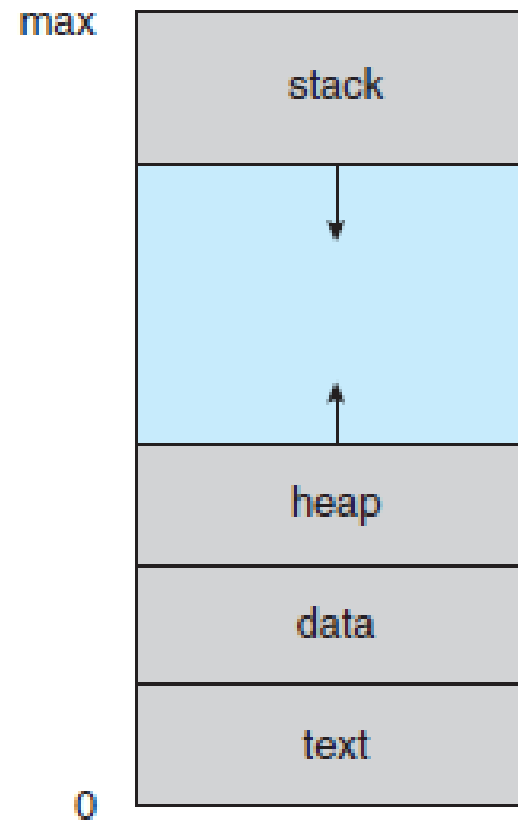
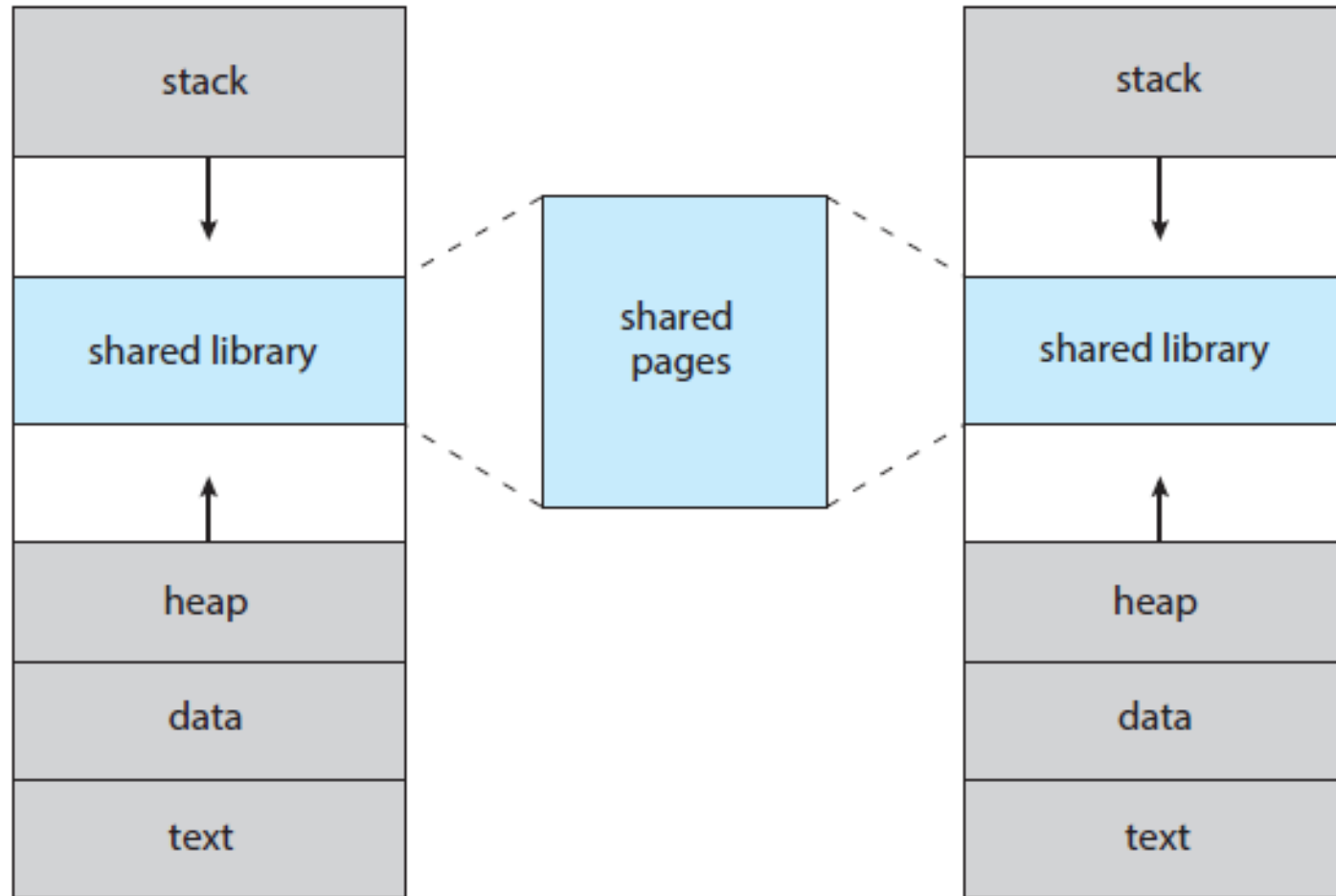


Figure: Virtual address space of a process in memory.

Shared Library using Virtual Memory



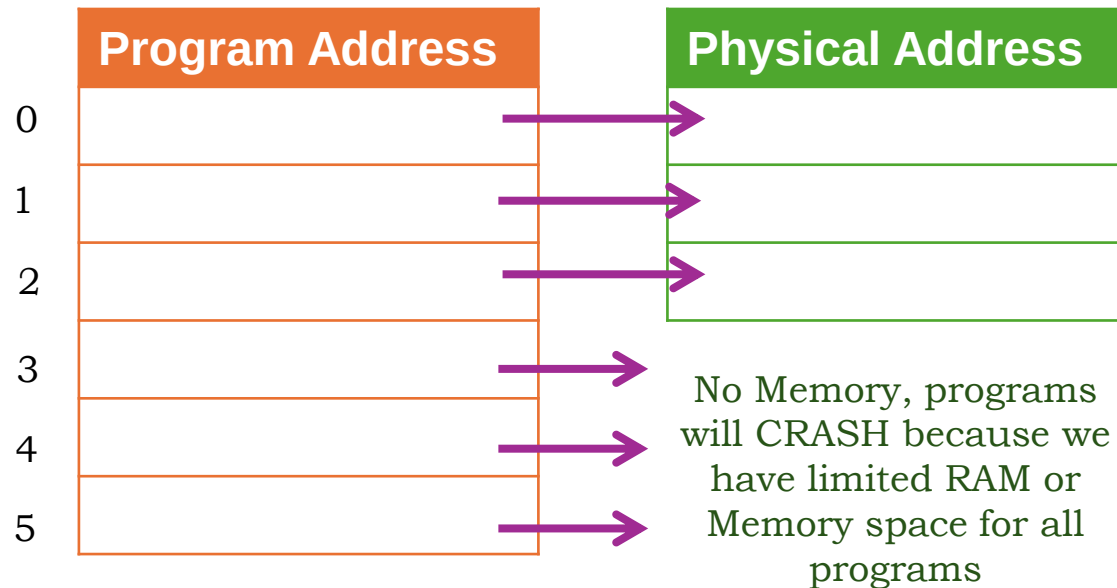
Virtual Memory - Concept

- Without Virtual Memory
- Program Address = RAM Address

- With Virtual Memory
- Use Virtual Memory (VM) to stop programs from crashing. VM takes program address and Maps it to the Physical Address. It provides Indirection.

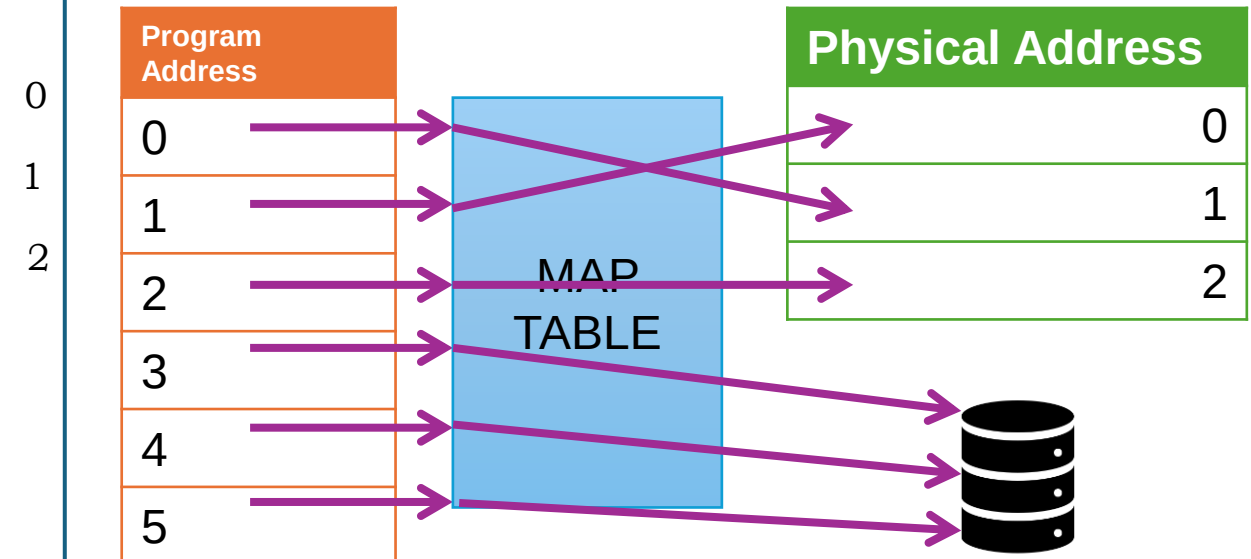
32-bit Program Address
Space (4 GB)

30-bit RAM Address
Space (1 GB)



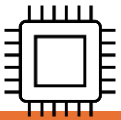
32-bit Program Address
Space (4 GB)

30-bit RAM Address
Space (1 GB)



How Virtual Memory Works

- Program Executes an instruction **load** specifying a Virtual Address Space.
- Translates this address to the physical address.



Program Virtual Address Space

ld, R3, 1024 (R0)

MAP Helps in Translation
VA -> PA

VA	PA
512	2
786	Disk
1024	2

Physical Address

0

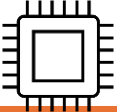
1

2

3

How Virtual Memory Works

- Program Executes an instruction load specifying a Virtual Address Space.
- Translates this address to the physical address.



Program Virtual Address Space
ld, R3, 1024 (R0)

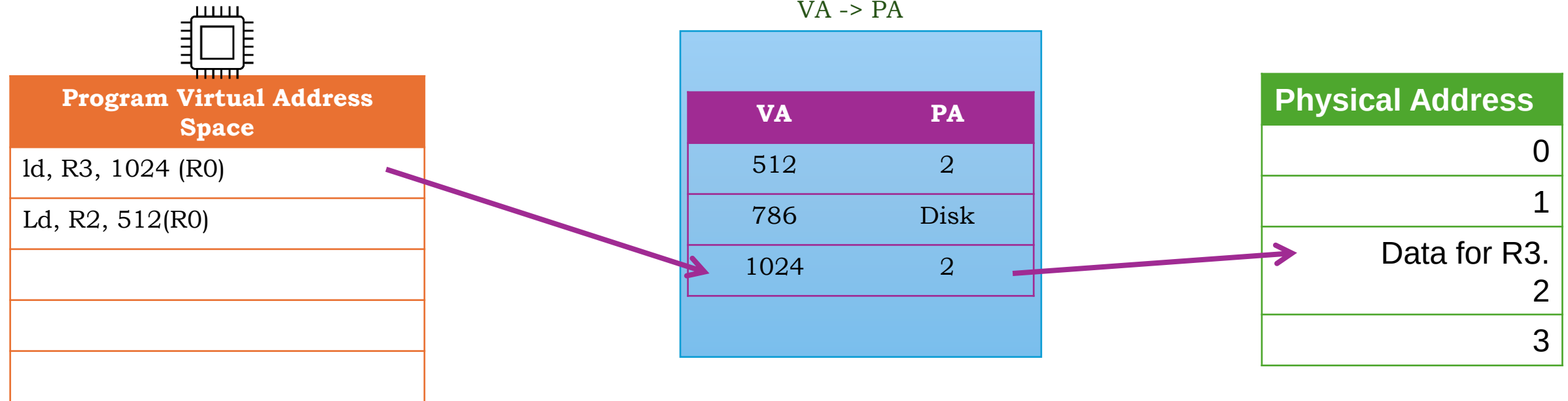
MAP Helps in Translation
VA -> PA

VA	PA
512	2
786	Disk
1024	2

Physical Address
0
1
2
3

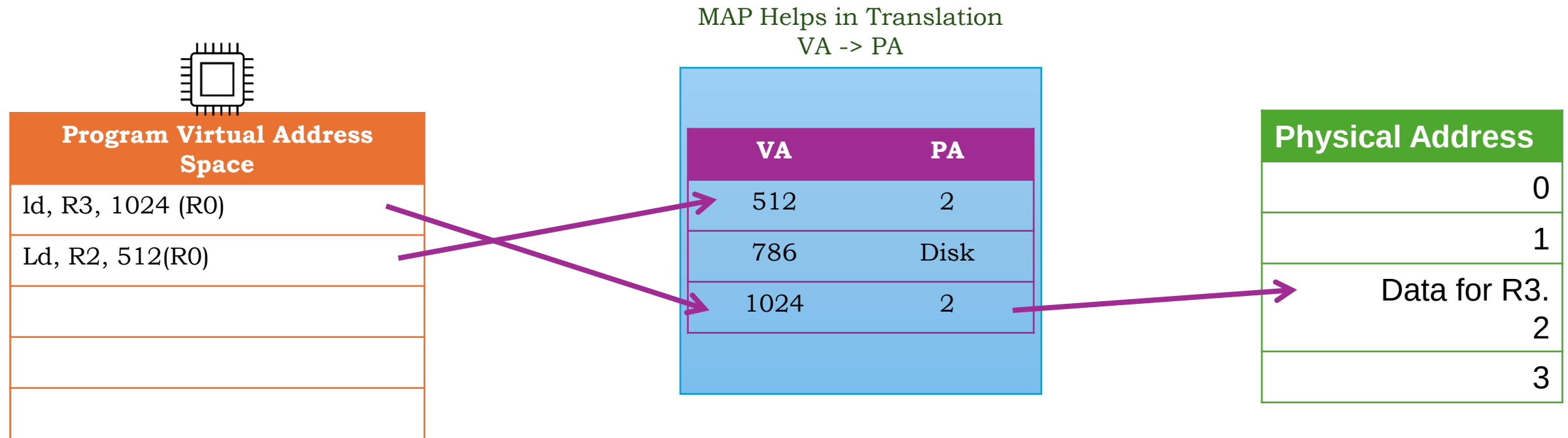
How Virtual Memory Works

- Program Executes an instruction load specifying a Virtual Address Space.
- Translates this address to the physical address.



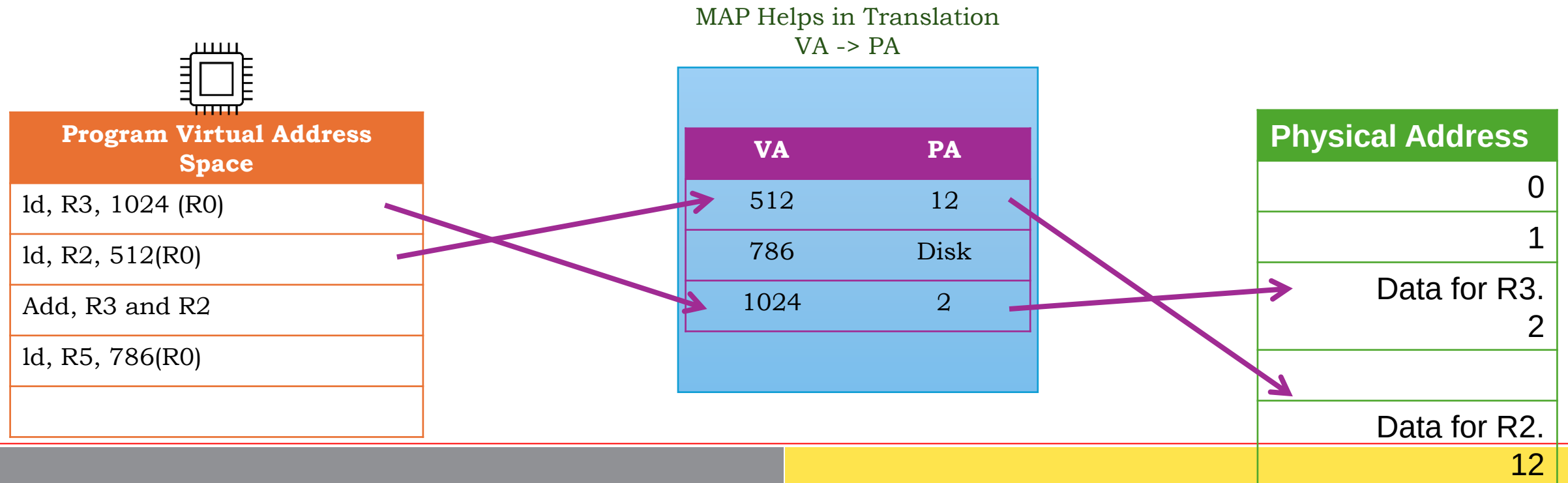
How Virtual Memory Works

- Program Executes an instruction load specifying a Virtual Address Space.
- Translates this address to the physical address.



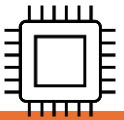
How Virtual Memory Works

- Program Executes an instruction load specifying a Virtual Address Space.
- Translates this address to the physical address.



How Virtual Memory Works

- Program Executes an instruction load specifying a Virtual Address Space.
- Translates this address to the physical address.
- If PA is not in memory, the OS loads it in from Disk, then updates Map Table. Then program reads from RAM.



Program Virtual Address Space
ld, R3, 1024 (R0)
ld, R2, 512(R0)
Add, R3, R2
ld, R5, 786(R0)

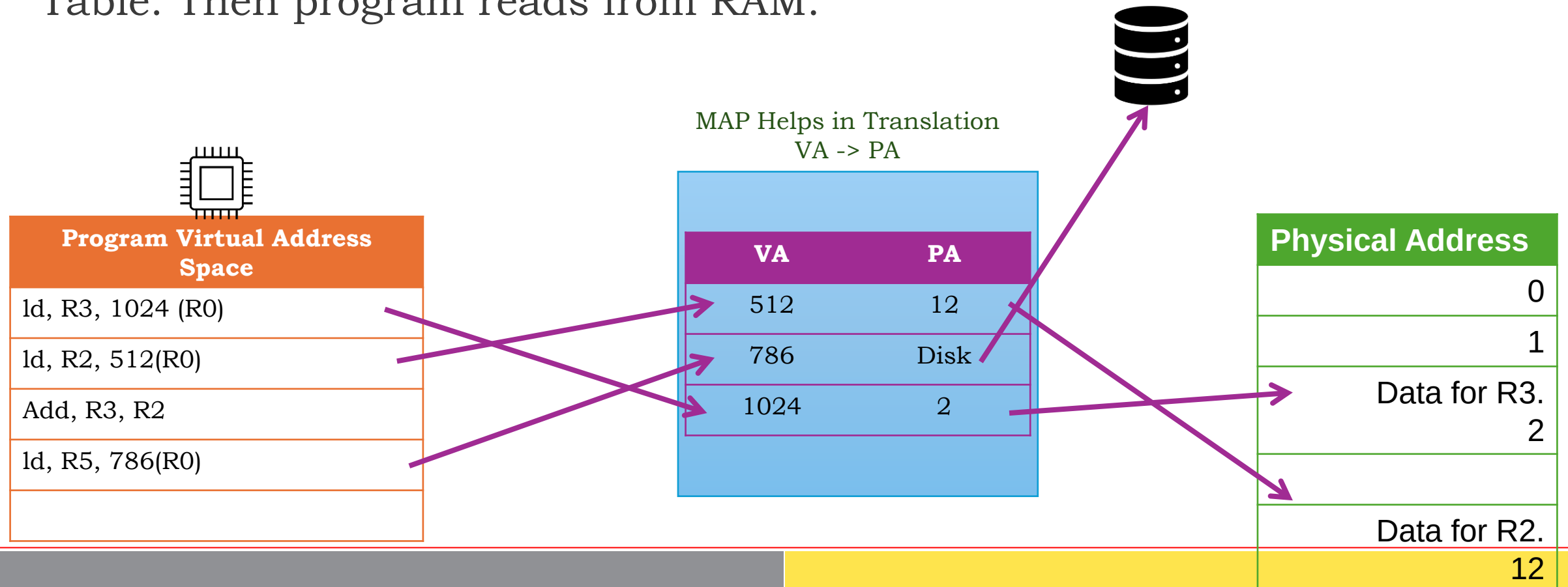
MAP Helps in Translation
VA -> PA

VA	PA
512	12
786	Disk
1024	2

Physical Address
0
1
Data for R3. 2
Data for R2. 12

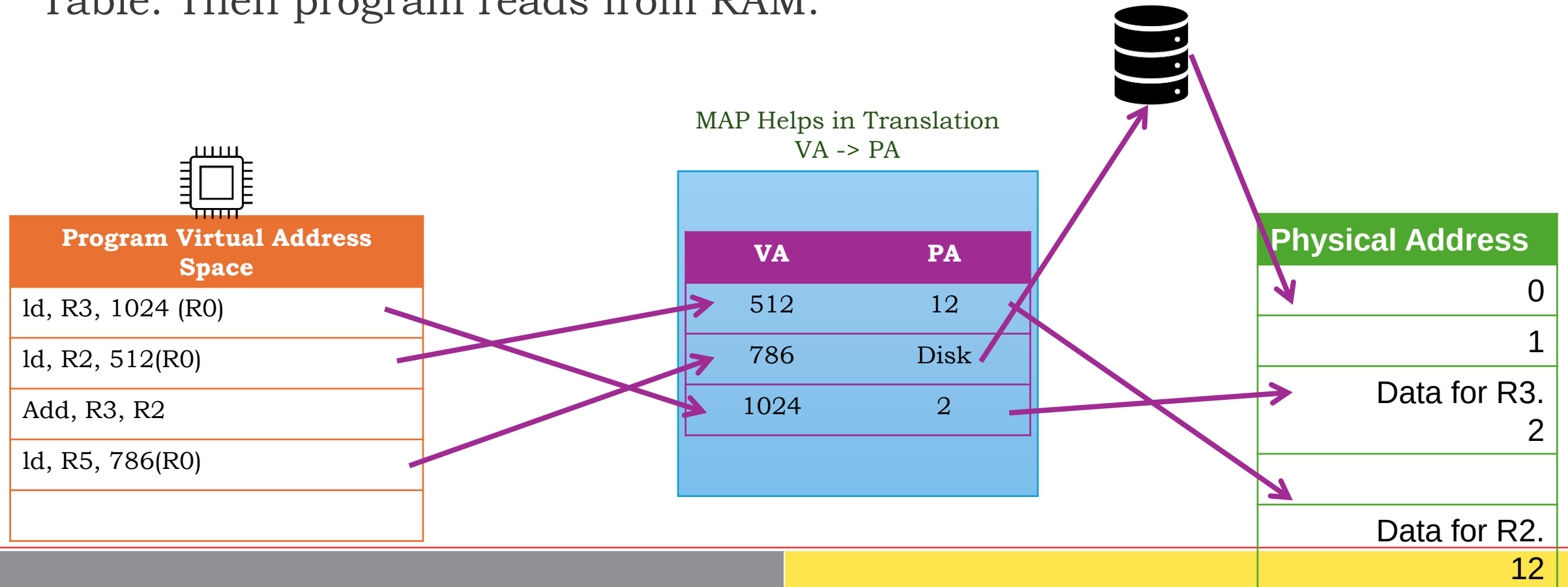
How Virtual Memory Works

- Program Executes an instruction load specifying a Virtual Address Space.
- Translates this address to the physical address.
- If PA is not in memory, the OS loads it in from Disk, then updates Map Table. Then program reads from RAM.



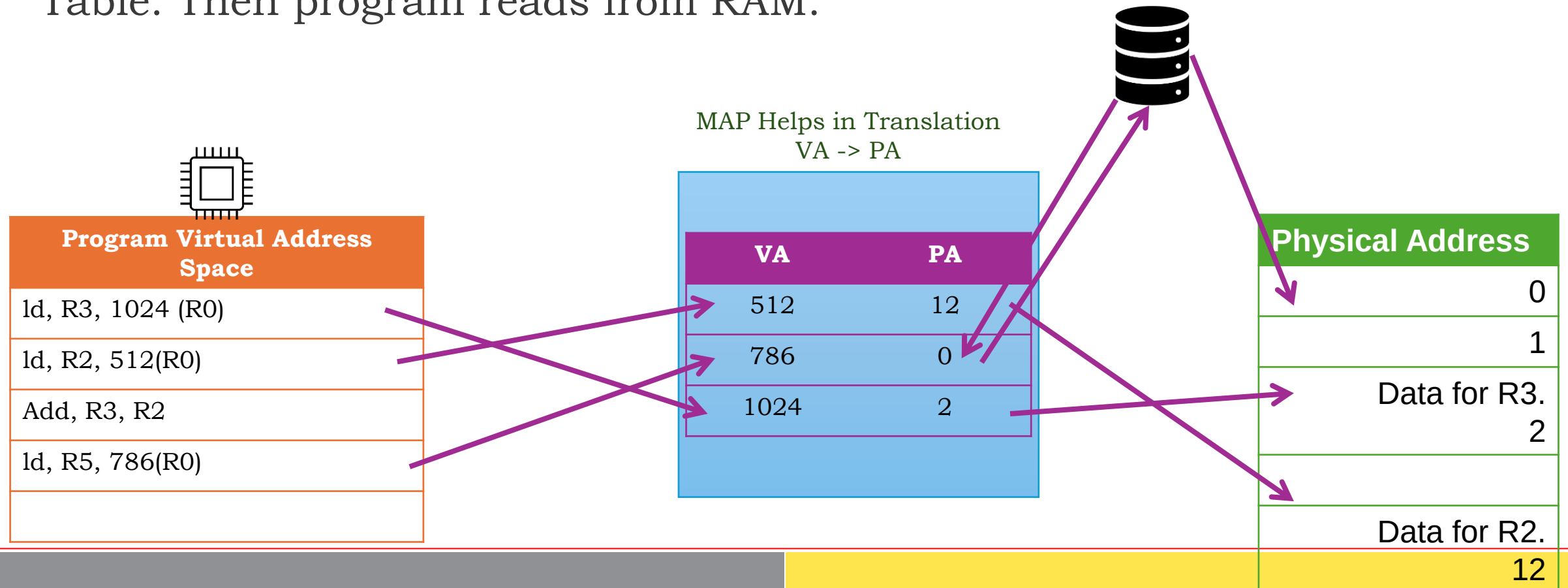
How Virtual Memory Works

- Program Executes an instruction load specifying a Virtual Address Space.
- Translates this address to the physical address.
- If PA is not in memory, the OS loads it in from Disk, then updates Map Table. Then program reads from RAM.



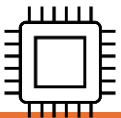
How Virtual Memory Works

- Program Executes an instruction load specifying a Virtual Address Space.
- Translates this address to the physical address.
- If PA is not in memory, the OS loads it in from Disk, then updates Map Table. Then program reads from RAM.



How Virtual Memory Works

- Program Executes an instruction load specifying a Virtual Address Space.
- Translates this address to the physical address.
- If PA is not in memory, the OS loads it in from Disk, then updates Map Table. Then program reads from RAM.



Program Virtual Address Space
ld, R3, 1024 (R0)
ld, R2, 512(R0)
Add, R3, R2
ld, R5, 786(R0)

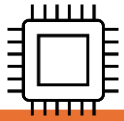
MAP Helps in Translation
VA -> PA

VA	PA
512	12
786	0
1024	2

Physical Address	
Data for R5.	0
	1
Data for R3.	2
Data for R2.	12

Page Table

- The Map from virtual addresses (VA) to Physical Addresses (PA) is the page Table.
- In our example, page table had one page table entry (PTE) for every virtual address.



Program Virtual Address Space
ld, R3, 1024 (R0)
ld, R2, 512(R0)
Add, R3, R2
ld, R5, 786(R0)

MAP Helps in Translation
VA -> PA

VA	PA
512	12
786	0
1024	2

Physical Address
Data for R5. 0
1
Data for R3. 2
Data for R2. 12

Page Table - Size

- How big is this Page table ? When each word (since virtual address are word access) has 1 entry ?
- It is 1 for every word so $2^{30} = 1$ billion entries.
- According to our example each entry needs the PA which is 32 bits so that's 1GB of memory for the Page Table alone.

Page Table Translation VA -> PA	
VA	PA
512	12
786	0
1024	2

Page Table - Size

- We need to translate every possible address.
 - Programs have **32 bit virtual address space**.
 - i.e. 2^{32} words that need page table entries (1 billion Entries)
 - If page table entry is not there then that program cannot be run or accessed in physical memory.
- Question is how making page table more manageable ?
 - Divide memory into chunks called Pages instead of words
 - Each Entry in Coarse grain handles 4kB of Data compared to 4B earlier as a word entry

Page Table Translation VA -> PA	
VA	PA
512	12
786	0
1024	2

FINE-GRAIN
Maps Each Word to
Physical Address
(as seen before)



Page Table Translation VA -> PA	
VA	PA
0 – 4095	4096 - 8191

COARSE-GRAIN
Maps Chunks (Pages) to
Main Memory

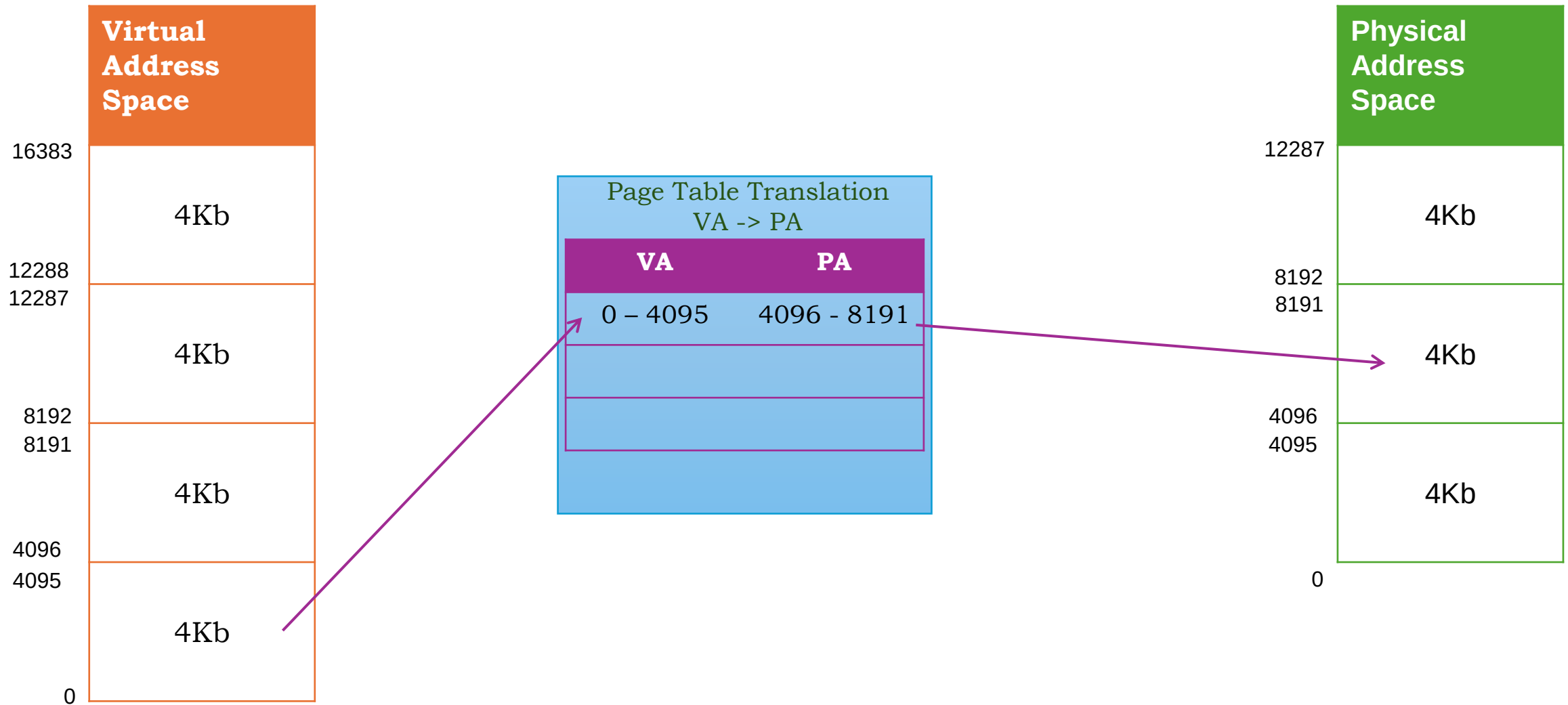
Page Table - Pages

- The Page Table Manages larger chunks (Pages) of Data:
 - Fewer page table entries needed to cover the whole address space
 - It is less flexibility, Need to move the complete page as it is in RAM before access.
 - Typically, 4Kb Pages (1024 words per page), so we need to have 1 billion words in 1-million-page table entry. That is close to 4 MB

Page Table Translation VA -> PA	
VA	PA
0 – 4095	4096 - 8191

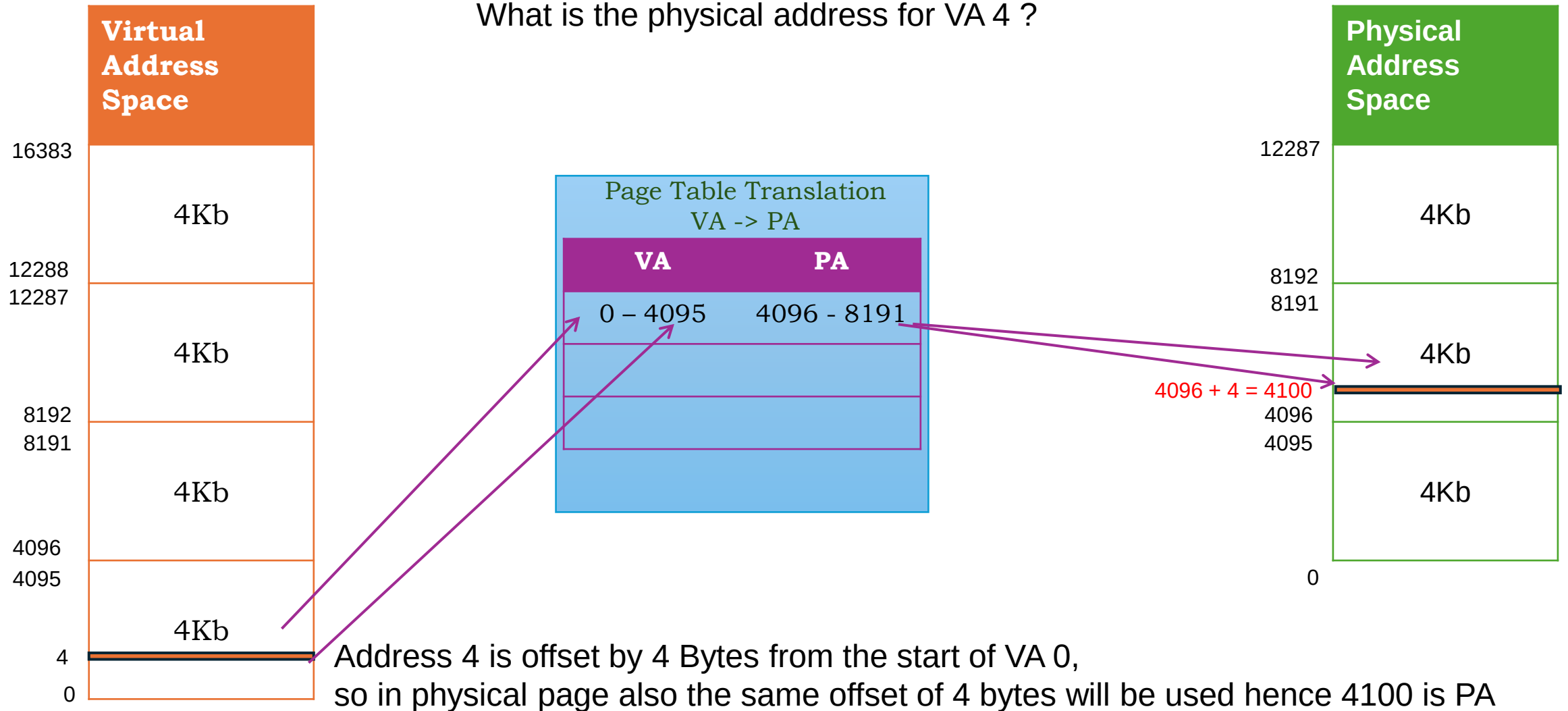
COARSE-GRAIN
Maps Chunks (Pages) to
Main Memory

Page Table - Pages - How do we Map



Page Table - Pages - How do we Map

What is the physical address for VA 4 ?



Address Translation

- Consider a system – 32-bit machine with 256MB of RAM and 4kB pages
 - 32-bit virtual address space – Each program can access 32-bit of address space. i.e. 4GB of virtual memory.
 - With respect to 256MB RAM, Physical Memory is 28-Bit address space.

Virtual Address Space (32 Bits)

4Kb
4Kb
4Kb
4Kb

Page Table Translation VA -> PA

VA	PA
0 – 4095	4096 - 8191

Physical Address Space (28 Bit)

4Kb
4Kb
4Kb

Address Translation

- Consider a system – 32-bit machine with 256MB of RAM and 4kB pages
 - Each PTE handles 4kB of address space.
 - For every page it is 4096 addresses (12bits) that don't get translated because it is offset.

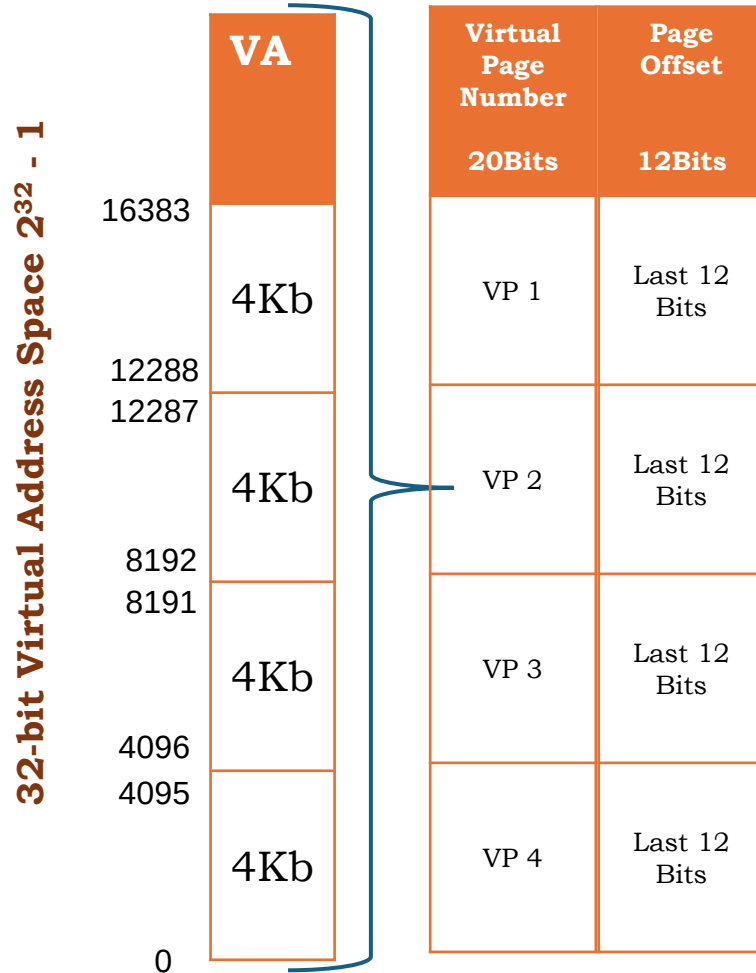
Virtual Address Space (20 Bits)	OFFSET (12 Bits)
4Kb	
4Kb	
4Kb	
4Kb	

Page Table Translation VA -> PA	
VA	PA
0 – 4095	4096 - 8191

Physical Address Space (16 Bit)	OFFSET (12 bits)
4Kb	
4Kb	
4Kb	

Address Translation

- Consider a system – 32-bit machine with 256MB of RAM and 4kB pages



Which Virtual Page Number?

What is Page offset within that page?

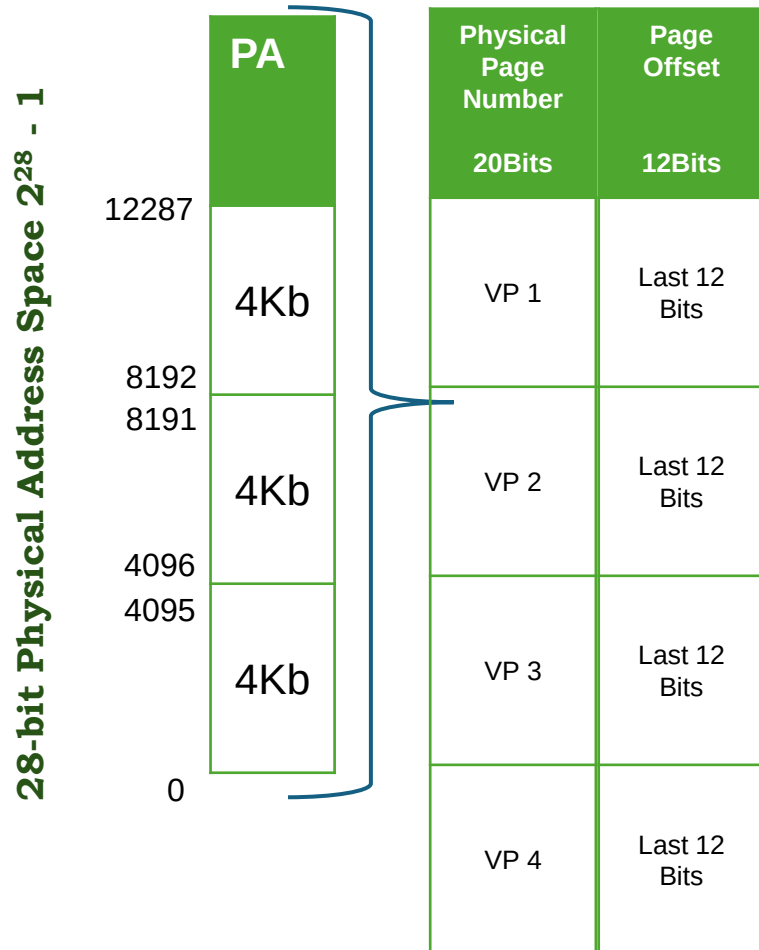
The other 20 bits of the address is Virtual Page Number (The upper bits)

+

The last 12Bits of the page is offset (The lower bits)

Address Translation

- Consider a system – 32-bit machine with 256MB of RAM and 4kB pages



Which Physical Page Number?

What is Page offset within that page?

The other 16 **bits** of the address is Physical Address

+

The last 12Bits of the page is offset (The lower bits)